

TD/TP 3 – Classes, invariants et RAI

D'une structure utilisable à un type responsable

MAM4 – Programmation C++

Durée : 2 heures, immédiatement après la séance 3.

Objectif. Partir de la `struct IntArray` de la séance 2 pour obtenir une classe qui protège ses données internes et libère automatiquement sa mémoire. Les sources initiales sont dans `code/seance3/`.

Organisation conseillée : 0–15 min : diagnostic et état correct ; 15–35 min : `class`, `private` et interface ; 35–65 min : constructeurs ; 65–75 min : rappel sur `const` ; 75–100 min : destructeur et tests ; 100–115 min : copie incorrecte ; 115–120 min : copie interdite et bilan.

```
g++ -std=c++20 -Wall -Wextra -pedantic int_array.cpp main.cpp -o int_array
```

1. Quand IntArray est-il dans un état correct ?

Compiler et exécuter le programme initial. Avant de modifier le type, écrire les règles qui doivent toujours rester vraies :

$$0 \leq \text{size} \leq \text{capacity}.$$

Si `capacity == 0`, alors `data == nullptr`. Si `capacity > 0`, `data` désigne une zone pouvant contenir `capacity` entiers. Les `size` premières cases sont les éléments présents. Ces règles sont les **invariants** de `IntArray`.

Dans `main.cpp`, essayer séparément trois modifications qui compilent avec la `struct` actuelle mais rendent l'objet incorrect, par exemple :

```
values.size = values.capacity + 1;
values.capacity = 2; values.size = 8;
values.capacity = 8; values.data = nullptr;
```

Ne pas conserver ces lignes dans la version finale.

2. Empêcher les modifications directes

Transformer le type en `class`. Rendre privées les données internes et adapter les fonctions de la séance précédente en méthodes membres. Il n'est pas nécessaire de réinventer les boucles de `sum` et `fill`.

```
class IntArray {
public:
    explicit IntArray(std::size_t capacity);
    ~IntArray();

    void push_back(int value);
    int at(std::size_t index) const;
    int sum() const;
    void fill(int value);
    std::size_t size() const;
    std::size_t capacity() const;

private:
    std::size_t capacity_{};
    std::size_t size_{};
    int* data_{nullptr};
};
```

Faire disparaître `initialize` et `destroy` de l'interface.

Question. Pourquoi ne voulons-nous plus écrire `IntArray values; initialize(values, 8);` en laissant exister, entre les deux lignes, un objet encore inutilisable ?

3. Construire uniquement des objets valides

Écrire le constructeur avec une liste d'initialisation. Une capacité nulle doit être refusée **avant de réserver la mémoire**. On pourra utiliser :

```
std::size_t checked_capacity(std::size_t capacity);
```

Cette fonction lance `std::invalid_argument` si `capacity == 0`. Tester la construction valide, puis la construction invalide séparément :

```
IntArray values{8};  
IntArray impossible{0}; // exception attendue
```

Question. Pourquoi écrire `explicit` devant le constructeur qui reçoit une capacité ?

Question. Une capacité nulle est-elle interdite dans tous les états internes, ou seulement lors de cette construction publique ?

Réponse attendue : le constructeur public la refuse ici, mais l'état vide `size_ = 0, capacity_ = 0, data_ = nullptr` reste un état valide de la classe.

4. Lire sans modifier

Compléter les méthodes. Vérifier que `size()` compile sur un objet constant, mais que `push_back()` est refusée :

```
const IntArray values{8};  
std::cout << values.size() << '\n'; // compile  
// values.push_back(3);             // ne compile pas
```

`const` après une méthode signifie : cette méthode ne modifie pas l'objet sur lequel elle est appelée. Dans `at`, un indice est invalide si `index >= size_` ; lancer alors `std::out_of_range`.

Question. Quel est le point commun entre `int sum(const IntArray& values);` et `int IntArray::sum() const; ?`

5. Libérer automatiquement la mémoire

Le destructeur doit exécuter `delete[] data_;`. Ajouter temporairement une trace dans le constructeur et dans le destructeur. Prédire l'ordre des quatre messages avant d'exécuter :

```
{  
    IntArray outer{4};  
    {  
        IntArray inner{8};  
    }  
}
```

Tester ensuite une sortie anticipée avec le code fourni :

```
void test_return(bool stop)  
{  
    IntArray values{8};  
    if (stop) return;  
    values.push_back(3);  
}
```

Puis observer une sortie par exception avec ce test minimal :

```
void test_exception()
{
    IntArray values{8};
    throw std::runtime_error{"test"};
}

try {
    test_exception();
} catch (const std::runtime_error& error) {
    std::cout << error.what() << '\n';
}
```

Compiler les tests avec des vérifications mémoire :

```
g++ -std=c++20 -Wall -Wextra -pedantic \
    -fsanitize=address,undefined -g \
    int_array.cpp main.cpp -o int_array
```

Les *sanitizers* repèrent de nombreuses erreurs mémoire pendant les tests ; leur silence ne prouve pas que tout programme possible est correct.

Question. Qui appelle maintenant `delete[]` lors d'une sortie de bloc ? Qui doit encore appeler `destroy(values)` ?

6. Que se passe-t-il si on copie IntArray ?

Avant d'interdire la copie, exécuter ce test avec AddressSanitizer :

```
{
    IntArray values{8};
    values.push_back(42);
    IntArray copy{values};
}
```

La copie automatique recopie l'adresse contenue dans `data_` :

```
values.data_ ----+
                +----> [ 42 | ... ]
copy.data_ -----+
```

Question. Les deux objets ont-ils chacun leur propre tableau ? Que se passe-t-il lorsque leurs deux destructeurs exécutent `delete[]` ?

Ajouter ensuite dans la partie publique de la classe :

```
IntArray(const IntArray&) = delete;
IntArray& operator=(const IntArray&) = delete;
```

Question. Pourquoi peut-on copier directement `size_` et `capacity_`, mais pas simplement copier `data_` ?

À retenir : copier un pointeur ne copie pas les données pointées.

À rendre. La classe `IntArray` non copiable, les tests exécutés avec les sanitizers et quelques lignes répondant aux questions suivantes : (1) comment le constructeur et le destructeur lient-ils la mémoire à la durée de vie de l'objet ? (2) pourquoi les champs privés aident-ils à maintenir `IntArray` dans un état correct ?

Bilan. Le constructeur crée un objet correct et réserve sa mémoire ; le destructeur la libère automatiquement. Les données internes sont privées. La copie automatique reste incorrecte : nous l'interdisons provisoirement.